PUSH BUTTON TRIGGERED TEMPERATURE DISPLAY SYSTEM

Project	Task 1 – Embedded Systems Internship
Platform	Arduino UNO
Components	TMP36 Sensor I Push Button I I2C 16x2 LCD
Interface	Adafruit_LiquidCrystal (I2C)
Date	April 2026

This report documents a button-triggered temperature monitoring system built on the Arduino UNO platform. A TMP36 analog sensor is read on demand when a push button is pressed, with the measured temperature displayed on a 16x2 I2C LCD. The document covers system design, hardware connections, working principles, annotated source code, expected outputs, and potential future improvements.

TASK -1

KEVIN VICTOR

Intern ID : CTIS8913

1. Introduction

Embedded systems frequently require human-initiated sensor readings rather than continuous polling, both to conserve power and to present data only when needed. This project implements a push-button-triggered temperature display using the Arduino UNO microcontroller, a TMP36 analog temperature sensor, and a 16×2 LCD driven over the I2C bus.

The system is intentionally minimal: the LCD shows an idle prompt until the operator presses a button, at which point the microcontroller samples the sensor, converts the raw ADC value to degrees Celsius, and updates the display. This design pattern — read on demand — is widely used in instrumentation and industrial HMI applications.

2. Objective

•Design and implement a button-triggered temperature monitoring system on Arduino UNO.

•Interface a TMP36 analog sensor and convert its output to a human-readable Celsius value.

•Display results on a 16×2 I2C LCD using the Adafruit_LiquidCrystal library.

•Apply edge detection (falling-edge) to reliably capture a single button press event.

•Produce clean, well-commented code suitable for professional review.

•Document the hardware, firmware, and operating procedure for reproducibility.

3. Components Used

Component	Qty	Role
Arduino UNO (ATmega328P)	1	Microcontroller – reads ADC, detects button events, drives LCD via I2C
TMP36 Analog Temperature Sensor	1	Outputs a voltage proportional to temperature (10 mV/°C). Connected to A0.
Push Button (momentary)	1	Operator input – triggers a temperature reading on each press. Connected to D2.
10 kΩ Resistor (optional)	1	External pull-up for extra noise immunity; not required when INPUT_PULLUP is used.
16×2 LCD with I2C backpack	1	Displays 'Press Button' at idle and 'Temp: XX.X C' after each press. Connected via SD
Jumper wires	~10	Point-to-point connections on breadboard or Tinkercad schematic.
Breadboard	1	Prototyping platform for component interconnection.

4. Circuit Design

All connections are made to the Arduino UNO header pins. The circuit requires no active power supply beyond the USB connection from a PC. The three subsystems — sensor, input, and display — are described below.

4.1 TMP36 Temperature Sensor

TMP36 Pin	Arduino Connection	Notes
Left (Vs / Power)	5 V	Sensor supply voltage
Middle (Vout)	A0 (Analog Input)	Analog signal: ~10 mV / °C
Right (GND)	GND	Common ground reference

Hold the TMP36 with its flat face towards you. The left pin is power, centre is the analog output, and right is ground. Reversing power and ground will damage the sensor.

4.2 16×2 I2C LCD

LCD Pin	Arduino Pin	Notes
GND	GND	Common ground
VCC	5 V	LCD and backlight power
SDA	A4	I2C data line
SCL	A5	I2C clock line

A4 and A5 on the UNO are the hardware I2C pins. The Adafruit_LiquidCrystal library communicates over this bus at the default address (0). If multiple I2C devices share the bus, ensure each has a unique address.

4.3 Push Button

Button Side	Arduino Pin	Notes
Side A	Digital Pin 2	Signal side – read by firmware
Side B	GND	Shorts Pin 2 to ground when pressed

5. Working Principle

5.1 INPUT_PULLUP and Idle State

The button pin is configured with **pinMode(BUTTON_PIN, INPUT_PULLUP)**. This activates the ATmega328P's internal ~20–50 kΩ pull-up resistor, tying the pin to 5 V through a resistor. With no button pressed, digitalRead returns **HIGH**. When the button is pressed, it creates a direct path from the pin to GND, pulling it to **LOW**. This inverted logic (unpressed = HIGH, pressed = LOW) is standard for pull-up configurations and eliminates the need for an external resistor.

5.2 Falling-Edge Detection

Detecting a button press — not a held state — requires edge detection. The firmware stores lastButtonState between loop iterations. A reading event fires only when **lastButtonState == HIGH** and **buttonState == LOW** simultaneously, meaning the transition HIGH → LOW just occurred. This prevents repeated reads while the button is held and ensures exactly one reading per press event.

5.3 ADC Reading and Voltage Conversion

The Arduino's 10-bit ADC maps 0–5 V to integer values 0–1023. The raw reading from analogRead(A0) is converted to voltage using:

```
voltage (V) = rawADC × (5.0 / 1023.0)
```

For example, a reading of 153 yields 153 × 0.004888 = 0.748 V.

5.4 Temperature Calculation

The TMP36 outputs 10 mV per degree Celsius with a 500 mV offset at 0 °C (per the datasheet). The simplified firmware formula used here is:

```
temperature (°C) = voltage × 100
```

This is adequate for ambient temperature readings. For precision applications, the full datasheet formula (temperature = (voltage – 0.5) × 100) should be applied to account for the 500 mV offset.

5.5 LCD Update Behaviour

On each button press, the LCD is cleared with lcd.clear() to remove the previous message. Row 0 (top) is then set to **"Temp:"** and Row 1 (bottom) to the numeric temperature followed by " C". A 300 ms delay prevents multiple triggers from a single mechanical press (debounce). The display retains the last reading indefinitely until the button is pressed again.

6. Code Explanation

The following is the complete annotated source code. The logic is identical to the original implementation; only comments and formatting have been improved for readability.

```
#include <Adafruit_LiquidCrystal.h>

// Pin assignments
const int TEMP_PIN = A0; // TMP36 analog output
const int BUTTON_PIN = 2; // Push button input

// LCD object - I2C address 0
Adafruit_LiquidCrystal lcd_1(0);

// Tracks previous button state for edge detection
int lastButtonState = HIGH;

void setup() {
pinMode(BUTTON_PIN, INPUT_PULLUP); // Internal pull-up
lcd_1.begin(16, 2); // 16 cols, 2 rows
lcd_1.print("Press Button"); // Idle message
}

void loop() {
int buttonState = digitalRead(BUTTON_PIN);
// Trigger on HIGH → LOW transition (falling edge)
if (lastButtonState == HIGH && buttonState == LOW) {
readAndDisplayTemp();
}
lastButtonState = buttonState;
}

void readAndDisplayTemp() {
int rawADC = analogRead(TEMP_PIN);
float voltage = rawADC * (5.0 / 1023.0);
float temperature = voltage * 100.0;

lcd_1.clear();
lcd_1.setCursor(0, 0);
lcd_1.print("Temp:");
lcd_1.setCursor(0, 1);
lcd_1.print(temperature);
lcd_1.print(" C");

delay(300); // Debounce delay
}
```

Key design decisions:

- **INPUT_PULLUP** – Avoids floating-pin noise without an external resistor.
- **Edge detection** – Fires exactly once per press regardless of hold duration.
- **lcd.clear()** – Prevents artefacts from shorter/longer previous strings.
- **delay(300)** – Simple mechanical debounce; sufficient for bench use.

•**voltage × 100** – Direct Celsius approximation from TMP36 output voltage.

7. Results

State / Event	LCD Row 0	LCD Row 1
Power-up / Reset	Press Button	(blank)
Button pressed (≈25 °C)	Temp:	74.69 C *
Button pressed (≈30 °C)	Temp:	80.57 C *
Button held (no re-fire)	Temp:	(unchanged)

* Values shown are from Tinkercad simulation. The simplified formula (voltage × 100) without the 0.5 V offset produces readings ~50 °C higher than actual ambient; applying the full formula would yield correct room-temperature values.

Demo Procedure (Tinkercad)

1. Open Tinkercad Circuits and recreate the schematic per Section 4.
2. Paste the code from code/temp_button.ino into the code editor.
3. Click **Start Simulation** and observe **"Press Button"** on the LCD.
4. Click the push button component — the LCD immediately updates to show **"Temp: XX.XX C"**.
5. Hover over the TMP36 and adjust the temperature slider; press the button again to see the updated reading.
6. Click the button and hold — the display does not re-fire (edge detection confirmed).

8. Possible Improvements

Continuous Monitoring Mode	Add a second button or long-press detection to toggle between on-demand and continuous update modes, refreshing the display every second.
Accurate TMP36 Formula	Apply the full conversion: temperature = (voltage − 0.5) × 100 to eliminate the ~50 °C systematic offset and produce correct ambient readings.
Buzzer Alert	Connect a passive buzzer to a PWM pin and trigger an audible alarm when temperature exceeds a configurable threshold.
Threshold Warning on LCD	Display a flashing " HIGH TEMP!" message on LCD row 0 when temperature surpasses a preset limit, keeping the numeric reading on row 1.
Interrupt-Driven Button	Attach the button to INT0 (Pin 2) and use attachInterrupt so the MCU can sleep in low-power mode and wake only on a press event.
IoT / Serial Logging	Add Serial.print() calls to stream timestamped temperature readings to a PC or pair with an ESP8266/ESP32 to push data to a cloud dashboard (e.g., ThingSpeak).

EEPROM Min/Max Logging	Store the session minimum and maximum temperatures in EEPROM so statistics survive a power cycle and can be displayed on a dedicated LCD screen.

9. Conclusion

This project successfully demonstrates a clean, event-driven design for embedded temperature monitoring. By triggering a sensor read only on a button press — rather than polling continuously — the firmware respects the principle of minimal, purposeful computation.

The use of INPUT_PULLUP simplifies the hardware by eliminating an external resistor, while falling-edge detection ensures exactly one reading per operator action. The Adafruit_LiquidCrystal I2C library reduces the wiring to just two signal lines (SDA/SCL), keeping the circuit clean and breadboard-friendly.

The system is fully functional in Tinkercad simulation and on real Arduino UNO hardware. It forms a solid baseline for the enhancements listed in Section 8, including threshold alerting, low-power sleep modes, and IoT connectivity.